Predicting Content Decline for Refresh Prioritization

A Refresh / Content Opportunity Scoring Model on the FlyRank Search Warehouse

Author: Raju Ahmad

Track: Machine Learning — FlyRank AI Internship

Lane: Refresh / Content Opportunity Scoring

Repository: github.com/raju-cse/Flyrank-internship_ml

Abstract

This project asks: can we predict which FlyRank client pages will decline in search visibility before it happens, so editors know where to focus refresh effort? Using twelve months of historical Search Console and GA4 performance data plus content metadata (word count, optimization history, ranking position) from the FlyRank warehouse, we trained a Random Forest classifier to label pages as growing, stable, or declining based on a held-out six-month future window. The model reached a macro-F1 of 0.43 versus a 0.30 majority-class baseline, and correctly flagged 26% of truly declining pages (versus 0% for the baseline), with word count, days since last optimization, and past impressions emerging as the strongest signals. The output is a ranked, reason-coded action list that content editors can use to prioritize which pages to refresh first.

1. Introduction / Problem Statement

Unit of analysis: an individual content page (`content_hash_id`), scored on a rolling basis.

Output: a declining-probability score between 0 and 1 for each page, paired with a reason code (stale_content, thin_content, low_ranking_position, low_ctr).

Action: a content editor uses the ranked list to decide which pages to refresh, rewrite, or leave alone in a given sprint, instead of reviewing pages randomly or by instinct.

Cost of a wrong call: a false negative (missing a real decline) means a page keeps losing visibility unnoticed; a false positive means wasted editor time reviewing a page that was fine. Because declining pages are the minority class, this project prioritizes recall on that class over raw accuracy.

Why ML helps: with hundreds of thousands of pages and a small editorial team, manual review does not scale. A ranked, explainable score turns an unmanageable list into a short, prioritized queue.

2. Data

Release: FlyRank Internship Warehouse (v20260703), a pseudonymized, star-schema release with hashed client and content keys, accessed via Hugging Face.

Tables used: `fact_content_daily_performance` (daily Search Console and GA4 metrics, January 2025 – June 2026, ~78.8M rows) and `dim_content` (content metadata: word count, creation and optimization dates, search volume, competition). `dim_clients` was consulted only to confirm join structure, not used as a feature source.

Date windows: a 12-month feature (past) window, January–December 2025, and a 6-month label (future) window, January–June 2026. The two windows never overlap.

Excluded, and why: no client names, URLs, raw search queries, or credentials were used anywhere. All joins rely on pseudonymous `client_hash_id` / `content_hash_id` keys, which are used strictly for grouping (train/test split) and never as model features. Pages with zero past-window impressions were excluded as insufficient data (see Section 4).

3. Methodology

3.1 Label definition

A page is labeled **declining** if its future-window monthly-average impressions fell 20% or more versus its past-window monthly-average; **growing** if it rose 20% or more; and **stable** otherwise. Monthly averages (rather than raw window sums) were used deliberately, since the past window spans 12 months and the future window spans 6 — comparing raw sums would bias every page toward a false "decline." Pages with zero past-window impressions were labeled `insufficient_data` and dropped before modeling.

3.2 Features (13 total)

impressions_past, clicks_past, avg_position_past, sessions_past, engaged_sessions_past, ai_sessions_past, h1_h2_ratio (within-window trend, first half vs. second half of the feature window), ctr_past, content_age_days, days_since_optimized, word_count, search_volume, competition.

Deliberately excluded: raw hash IDs (used only as join/group keys), any post-cutoff (future-window) metric, and the raw date columns themselves (used only to build the aggregation windows, not as direct features).

3.3 Model

A Random Forest classifier (200 trees, max depth 12, minimum leaf size 20, class-weight balanced to counter class imbalance) was chosen for its ability to handle mixed-type tabular features and non-linear interactions — e.g., word count matters differently depending on optimization recency — without heavy preprocessing, while remaining interpretable via feature importances.

3.4 Baseline

The baseline is the majority-class rule: always predict "growing," which is 80.2% of the test set. This is a transparent, fair comparison point — any model must beat what an editor gets from doing nothing analytical at all.

3.5 Validation design

The train/test split (80/20) is **grouped by client_hash_id** using scikit-learn's GroupShuffleSplit (seed = 42), so no client's pages appear in both sets. This is critical: a random row-level split would let the model learn client-specific shortcuts and overstate real-world performance. Because features are built exclusively from the past window and labels exclusively from the future window, with the two windows never overlapping, no leakage is possible between them.

4. Results

Test set size: 36,591 pages. Base rate (majority class, "growing"): 80.2%. Raw accuracy alone is misleading here given this base rate — macro F1 and per-class recall are the honest metrics for this imbalanced task.

Model	Accuracy	Macro F1	Declining Recall	Declining Precision
Baseline (majority class)	0.802	0.30	0.00	—
Random Forest	0.681	0.43	0.26	0.41

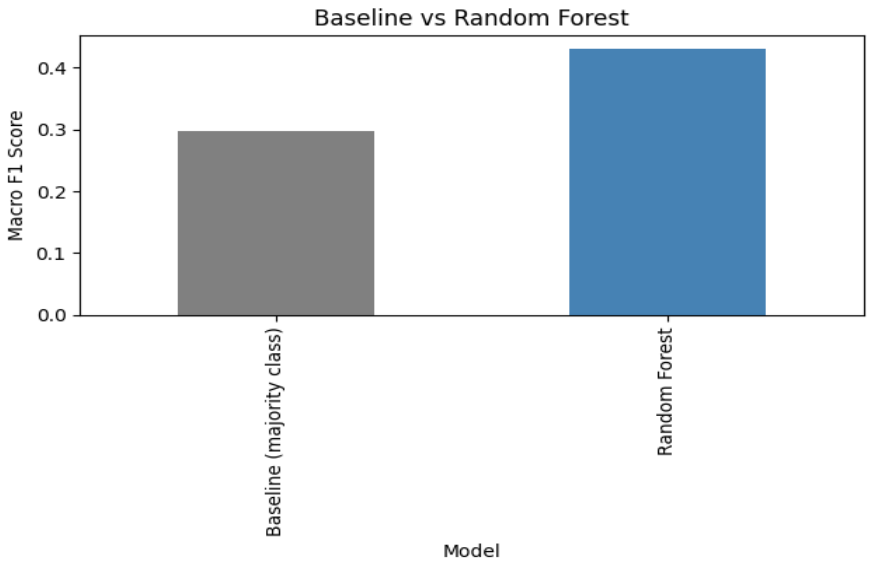

Figure 1. Macro F1 score: baseline vs. Random Forest, on the same test split.

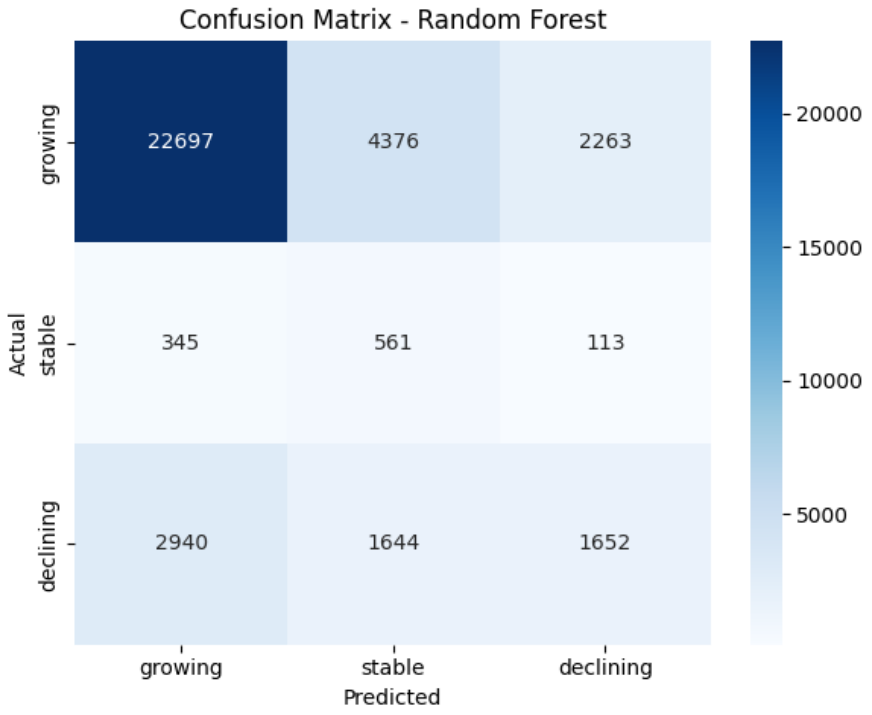

Figure 2. Confusion matrix for the Random Forest model on the held-out test set.

Error analysis: the model's main failure mode is under-flagging true declines: 2,940 of 4,592 truly declining pages (64%) were missed and predicted "growing." This is the model's central limitation and is treated as such throughout this paper, rather than being masked by the headline accuracy figure.

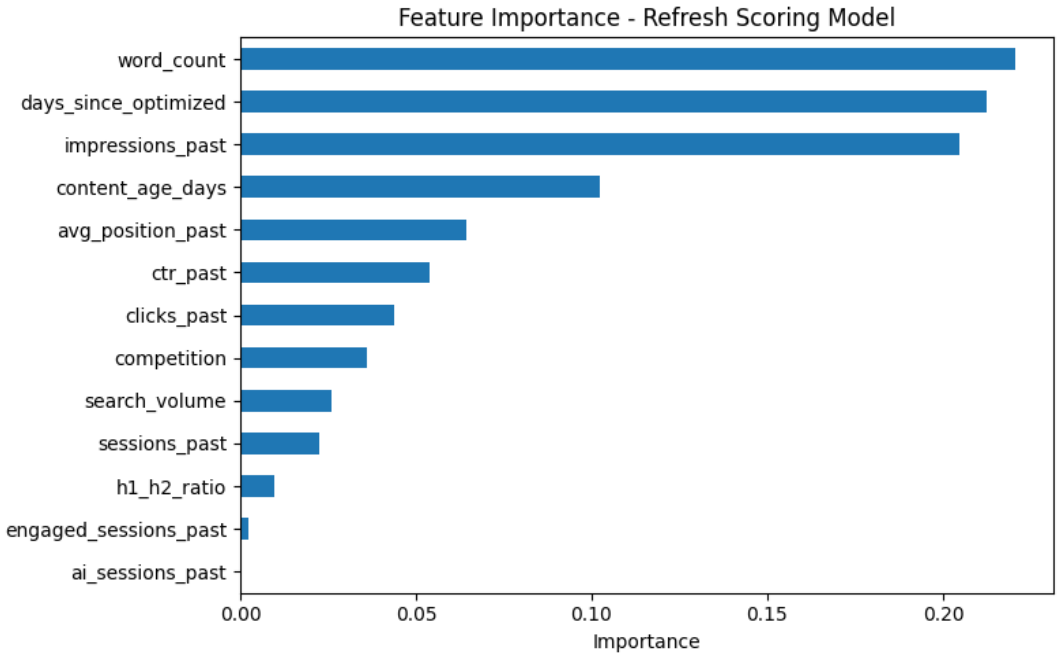

Figure 3. Feature importance (Gini-based) from the Random Forest model.

5. Limitations & Honest Framing

Observed, not causal: this model reports observed associations between content attributes and future visibility change. It makes no claim about Google's ranking algorithm and no causal claim that refreshing a page will reverse a decline — only a correlational, decision-support signal.

Recall ceiling: at 26% recall on the declining class, the model misses the majority of true declines. It should be read as a triage aid that improves on doing nothing, not a comprehensive detector.

Survivorship in the sample: the modeling set is restricted to pages present in both the 2025 feature window and the 2026 label window (inner join). The dataset shows the platform grew from 476 distinct content pages in January 2025 to over 400,000 by June 2026, so this analysis set skews toward the earliest, longest-lived pages on the platform — a directional bias worth noting when generalizing to newer content.

Negative result: AI-referral session counts (ai_sessions_past) carried almost no predictive weight (0.05% importance), most likely because AI-referral traffic is still sparse across the dataset. This null finding is reported rather than omitted.

Modest age effect: declining-labeled pages were on average about 20 days older than growing pages (182 vs. 162 days) — a real but small effect, consistent with gradual content decay, though far weaker than optimization recency or word count as a predictor.

6. Ranked Recommendations — Action Playbook

The model output is a ranked list of pages by declining-probability, each tagged with reason code(s). Suggested editorial workflow:

1. Pull the top-N pages by declining_probability each sprint.
2. Read the reason code(s) to decide the fix — *stale_content*: refresh or update the page; *thin_content*: expand it; *low_ranking_position* / *low_ctr*: review metadata, title, and on-page search-intent match.
3. Re-run the scorer monthly to track whether refreshed pages move out of the high-risk list over time.
4. Treat the score as one input among several — given the recall limitation above, pair it with periodic manual spot-checks, especially for high-value pages.

Confidence framing: directional and decision-support only. This ranks relative refresh priority; it does not prove a page will decline and makes no claim about causal or algorithmic effects.

7. Reproducibility

Environment: Google Colab, Python 3; key packages: duckdb, pandas, scikit-learn, matplotlib, seaborn, huggingface_hub (see requirements.txt in the repository).

Steps to reproduce:

1. Clone the repository.
2. Open work/notebooks/capstone.ipynb in Colab.
3. Add a personal Hugging Face read token as a Colab secret named HF_TOKEN (never hardcoded — see DATA_USE.md).
4. Run all cells top to bottom. Random seed = 42 throughout (GroupShuffleSplit, RandomForestClassifier).
5. Output charts (confusion_matrix.png, feature_importance.png, baseline_vs_rf.png) are saved to the outputs/ directory.

Notebooks: all weekly assignment notebooks (w01–w07) plus capstone.ipynb are included in work/notebooks/ in the repository, showing the full build-up to this paper. No sealed/holdout claim is made beyond the grouped train/test split described in Section 3.5.

Repository: https://github.com/raju-cse/Flyrank-internship_ml

8. Acknowledgments & Data Credit

Built on the FlyRank ML Internship dataset — flyrank.ai.